CS 189/289

Today's lecture outline

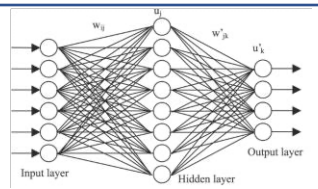
1. Recap of Neural Networks
2. Backpropagation for neural networks

Assigned reading:

Chapter 8 (Backpropagation): Intro-8.1.4, 8.2 (up to but not including 8.2.1 and beyond)

Overall set-up is same as before

- Training Data: $S = \{(x_i, y_i)\}_{i=1}^{N}$ $\quad x \in R^D$ $\quad y \in \{-1, +1\}$

- Model Class: $f(x|w) =$ 

Composition of functions

- Loss Function: $LL(w) = \sum_i \log p(y_i|x_i, w)$ Log Likelihood

- Learning Objective: $\hat{w} = argmin_w \{-LL(w)\}$ Optimization Problem

gradient descent

$\mathbf{w}^{(0)} = \text{Initialize}()$
for τ in range$(1, \tau_{\max})$:
  $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla J(\mathbf{w}^{(\tau-1)})$
  if Converged$(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$: terminate

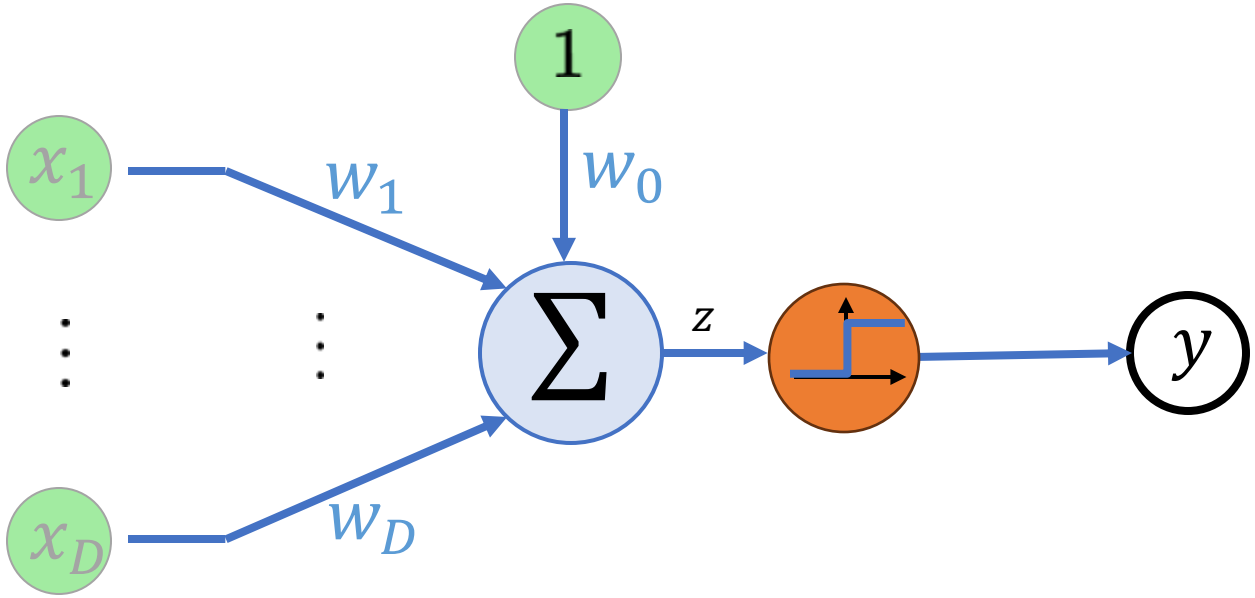
$J(w) = -LL(w)$

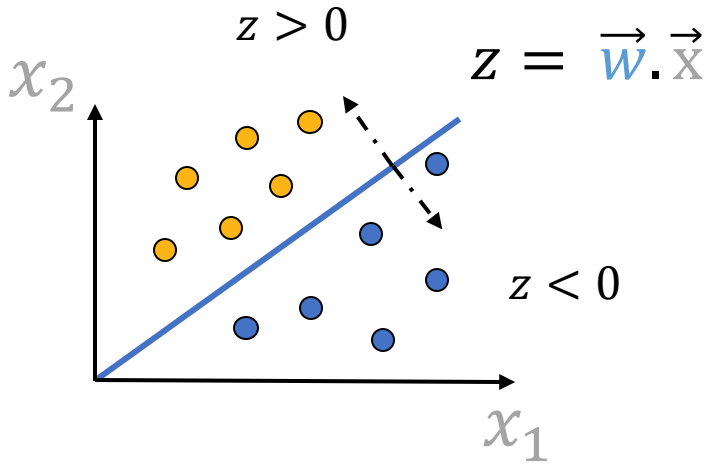
Recall: A Neuron with Step Function as Activation

Assume the activation function to be a **step function:**

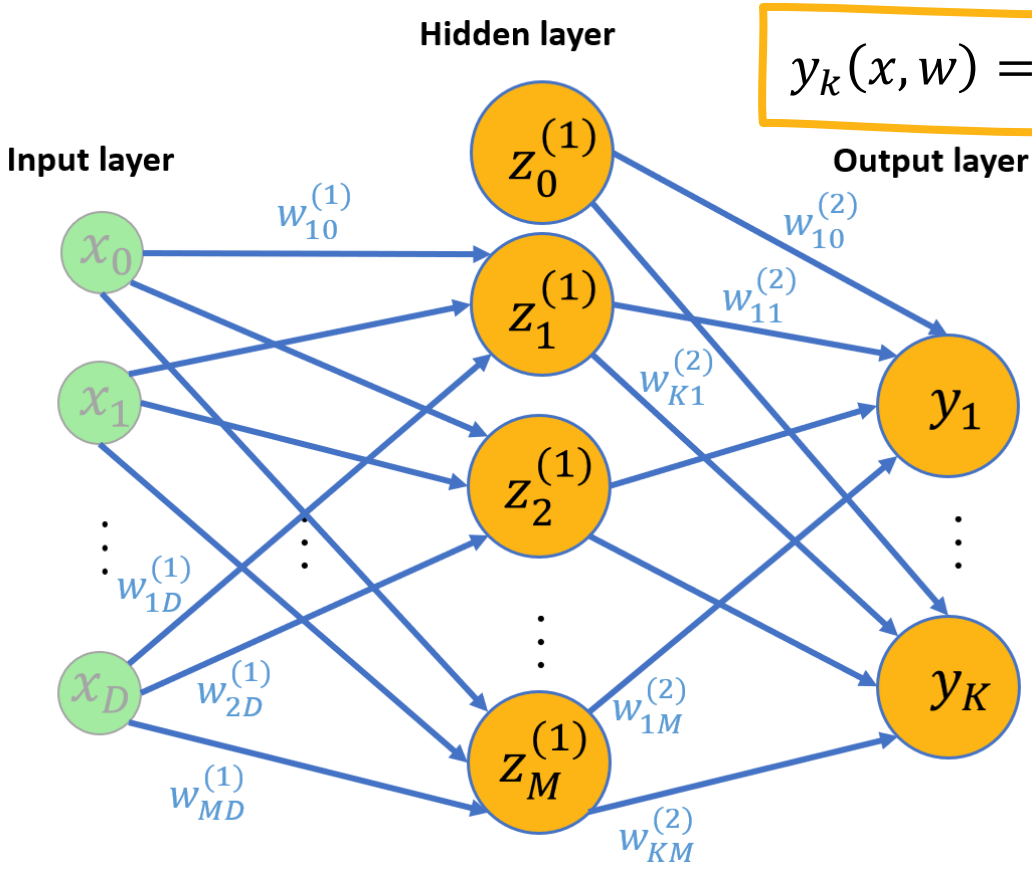
- If the weighted combination of inputs, z, is **negative**, the output is **0** → neuron is not activated.
- If the weighted combination of inputs z, is **positive**, the output is **1** → neuron is activated.

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$





Recall: neural networks are just function composition



$$y_k(x, w) = f\left(a_k^{(2)}\right)$$

$$k \in \{1, \ldots, K\}$$

$$f\left(a_k^{(2)}\right) = f\left(\sum_{j=1}^{M} w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)}\right)$$

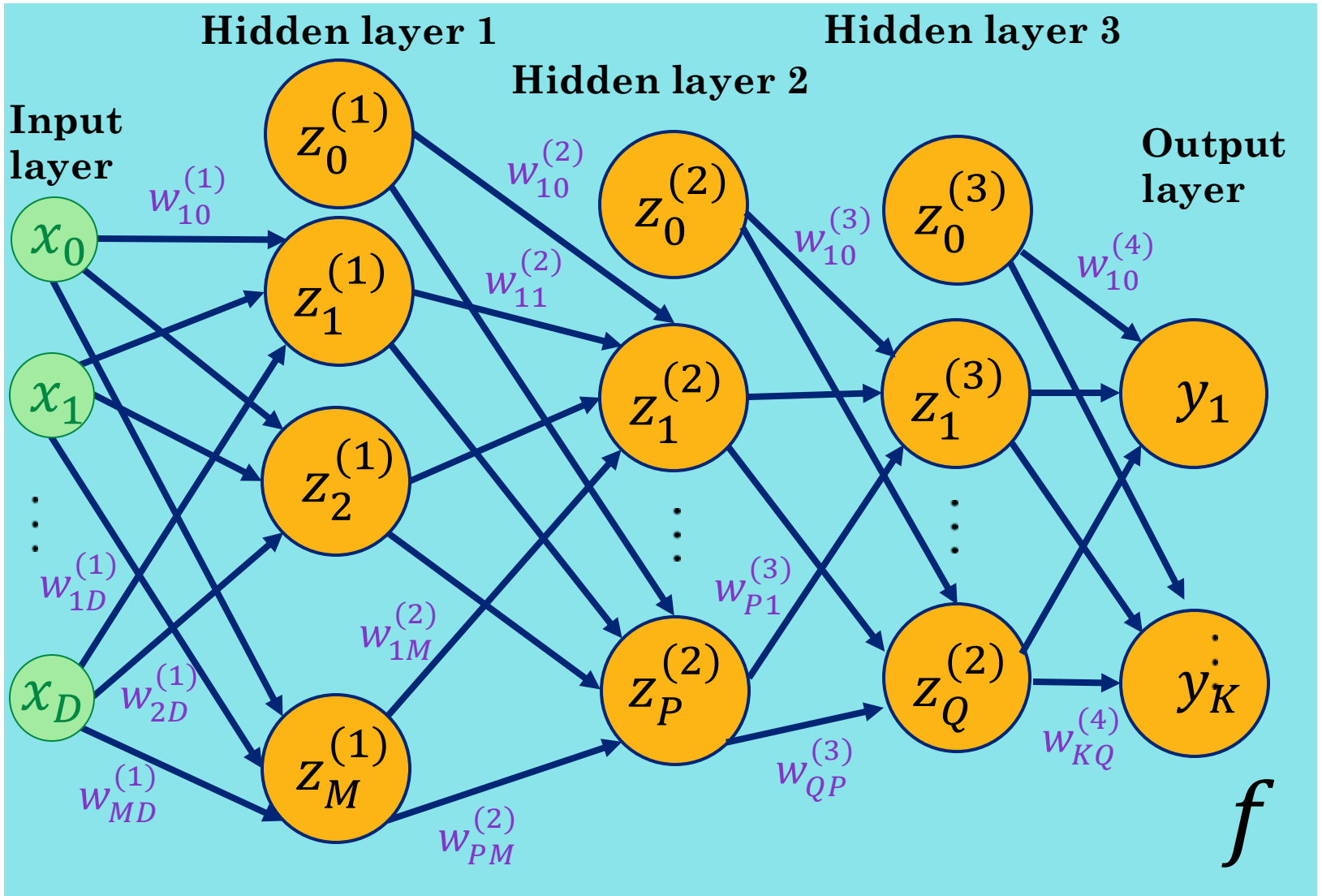
$$= f\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$$

$$= f\left(W^{(2)} h\left(W^{(1)} \mathrm{x}\right)\right)$$

$$z_j^{(1)} = h(a_j^{(1)}) = h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right)$$

$$j \in \{1, \ldots, M\}$$

Recall: Deep Neural Network



$l \in \{1, \ldots, L\}$

Activation at layer l

$$z^{(l)} = h^{(l)}\left(W^{(l)} z^{(l-1)}\right)$$

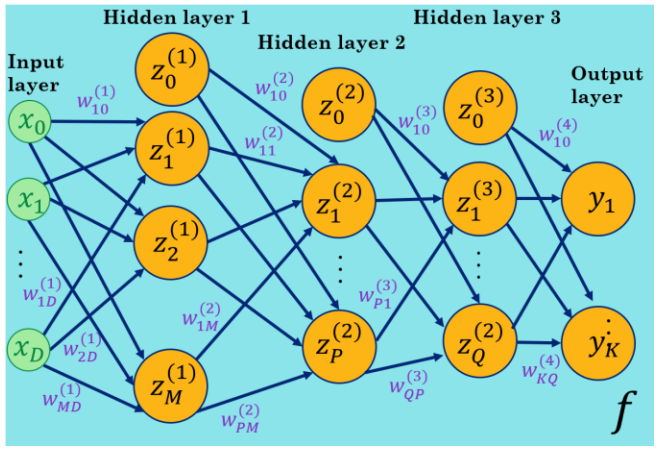
Activation function in layer l

Weights in layer l

$$z^{(0)} = \mathrm{x}$$

$$z^{(L)} = \mathrm{y}$$

Recall: Some Activation Functions

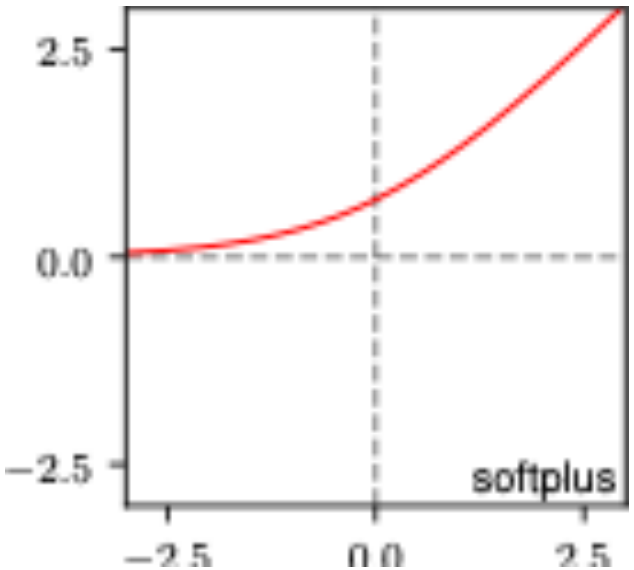


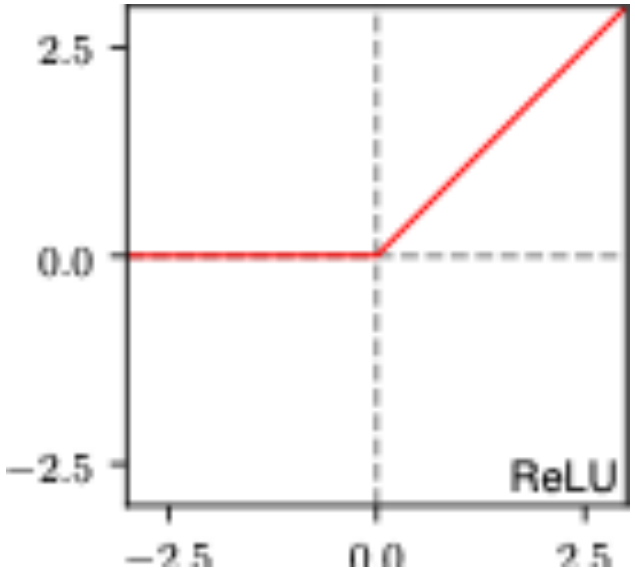
Activation at layer l

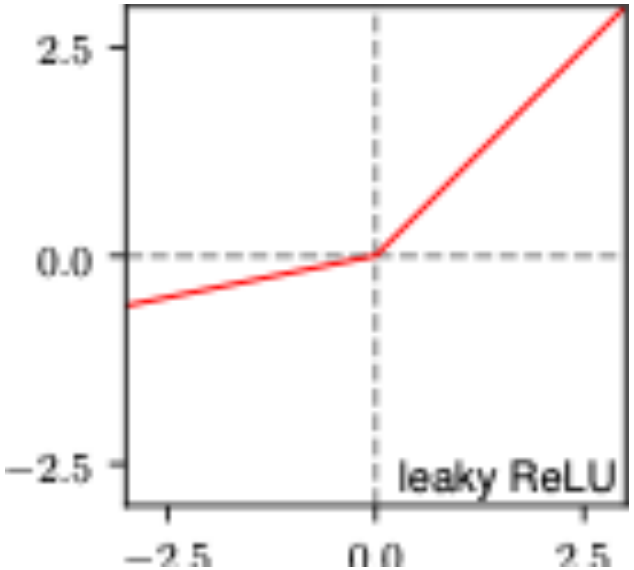
$$z^{(l)} = h^{(l)}\left(W^{(l)} z^{(l-1)}\right)$$

Activation function in layer l

Weights in layer l







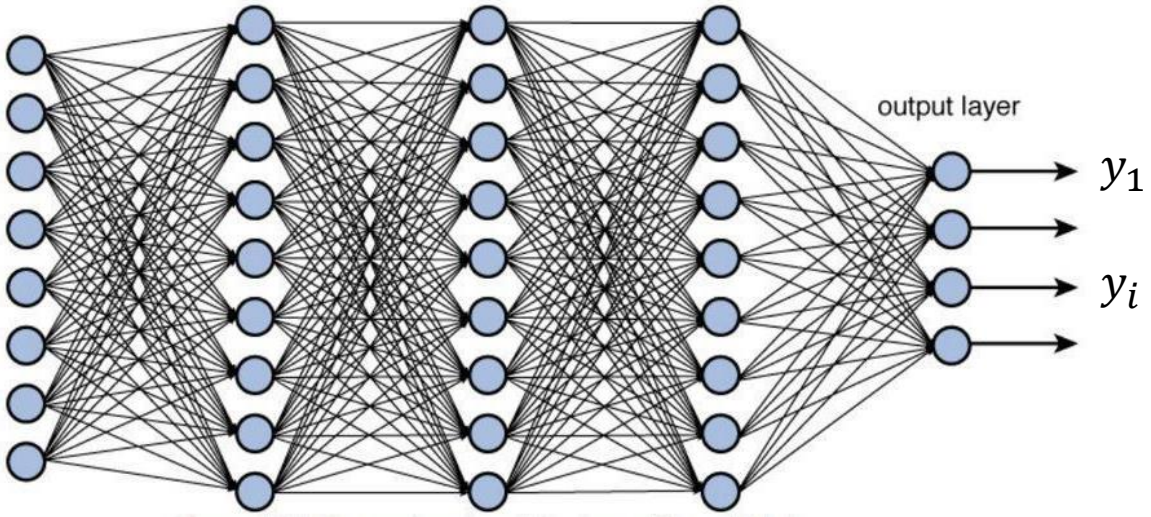
$$h(a) = \ln(1 + \exp(a))$$

$$h(a) = \max(0, a)$$

$$h(a) = \max(0, a) + \alpha \min(0, a)$$

How do we train a neural network?

- Typically with Stochastic Gradient Descent and flourishes (e.g., *Adam—see L13, Bishop book 7.3.3*).
- Isn't it going to be super complicated to take gradients of these huge networks?
- Yes, but *backpropagation* is the recipe for this.
- Next: calculus chain rule → backprop.



$$y_i = h_L(W^{(L)} \ldots h_2\left(W^{(2)} h_1(W^{(1)} \mathrm{x})\right)$$

Calculus: Univariate chain rule

if $f(x)$ and $x(t)$ are univariate functions, then

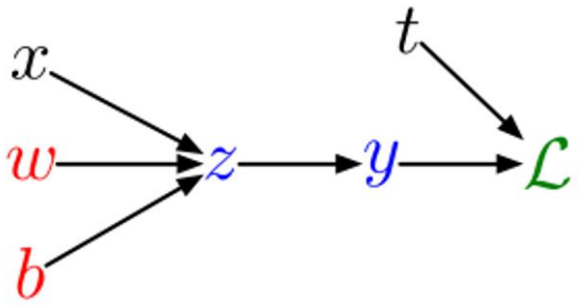
$$\frac{\mathrm{d}}{\mathrm{d}t} f(x(t)) = \frac{\mathrm{d}f}{\mathrm{d}x} \cdot \frac{\mathrm{d}x}{\mathrm{d}t}.$$

Following backprop slides adapted from Roger Grosse: http://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/

A simple non-linear regression example

(assume all scalars)

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$



For gradient descent/SGD, need to compute $\frac{\partial \mathcal{L}}{\partial w}$ *and* $\frac{\partial \mathcal{L}}{\partial b}$

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

How you would have done it in calculus class

$$\mathcal{L} = \frac{1}{2}(\sigma(wx + b) - t)^2$$
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2}(\sigma(wx + b) - t)^2\right]$$
$$= \frac{1}{2}\frac{\partial}{\partial w}(\sigma(wx + b) - t)^2$$

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

How you would have done it in calculus class

$$\mathcal{L} = \frac{1}{2}(\sigma(wx + b) - t)^2$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2}(\sigma(wx + b) - t)^2\right]$$

$$= \frac{1}{2}\frac{\partial}{\partial w}(\sigma(wx + b) - t)^2$$

$$= (\sigma(wx + b) - t)\frac{\partial}{\partial w}(\sigma(wx + b) - t)$$

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

How you would have done it in calculus class

$$\mathcal{L} = \frac{1}{2}(\sigma(wx + b) - t)^2$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2}(\sigma(wx + b) - t)^2\right]$$

$$= \frac{1}{2}\frac{\partial}{\partial w}(\sigma(wx + b) - t)^2$$

$$= (\sigma(wx + b) - t)\frac{\partial}{\partial w}(\sigma(wx + b) - t)$$

$$= (\sigma(wx + b) - t)\sigma'(wx + b)\frac{\partial}{\partial w}(wx + b)$$

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

How you would have done it in calculus class

$$\mathcal{L} = \frac{1}{2}(\sigma(wx + b) - t)^2$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2}(\sigma(wx + b) - t)^2\right]$$

$$= \frac{1}{2}\frac{\partial}{\partial w}(\sigma(wx + b) - t)^2$$

$$= (\sigma(wx + b) - t)\frac{\partial}{\partial w}(\sigma(wx + b) - t)$$

$$= (\sigma(wx + b) - t)\sigma'(wx + b)\frac{\partial}{\partial w}(wx + b)$$

$$= (\sigma(wx + b) - t)\sigma'(wx + b)x$$

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

How you would have done it in calculus class

$$\mathcal{L} = \frac{1}{2}(\sigma(wx + b) - t)^2$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2}(\sigma(wx + b) - t)^2\right]$$
$$= \frac{1}{2}\frac{\partial}{\partial w}(\sigma(wx + b) - t)^2$$
$$= (\sigma(wx + b) - t)\frac{\partial}{\partial w}(\sigma(wx + b) - t)$$
$$= (\sigma(wx + b) - t)\sigma'(wx + b)\frac{\partial}{\partial w}(wx + b)$$
$$= (\sigma(wx + b) - t)\sigma'(wx + b)x$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial}{\partial b}\left[\frac{1}{2}(\sigma(wx + b) - t)^2\right]$$
$$= \frac{1}{2}\frac{\partial}{\partial b}(\sigma(wx + b) - t)^2$$
$$= (\sigma(wx + b) - t)\frac{\partial}{\partial b}(\sigma(wx + b) - t)$$
$$= (\sigma(wx + b) - t)\sigma'(wx + b)\frac{\partial}{\partial b}(wx + b)$$
$$= (\sigma(wx + b) - t)\sigma'(wx + b)$$

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

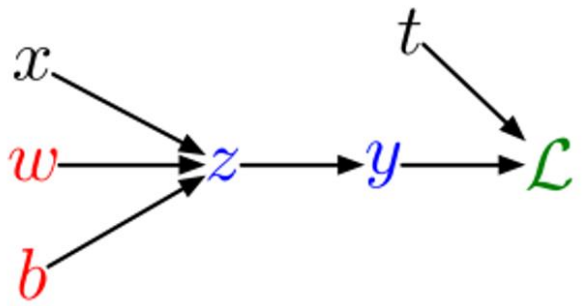
Instead, let's work *backwards*, systematically from the loss, computing each intermediate derivative as we go:

Computing the loss:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Computing the derivatives:

$$\frac{d\mathcal{L}}{dy} = y - t$$
$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy} \cdot \sigma'(z)$$



Remember, the goal isn't to obtain closed-form solutions, but to be able to write a program that efficiently computes the derivatives.

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

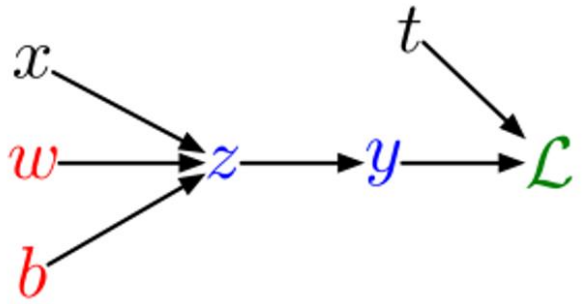
Instead, let's work *backwards*, systematically from the loss, computing each intermediate derivative as we go:

Computing the loss:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Computing the derivatives:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} = y - t$$
$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} \cdot \sigma'(z)$$
$$\frac{\partial\mathcal{L}}{\partial w} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} \cdot x$$



Remember, the goal isn't to obtain closed-form solutions, but to be able to write a program that efficiently computes the derivatives.

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

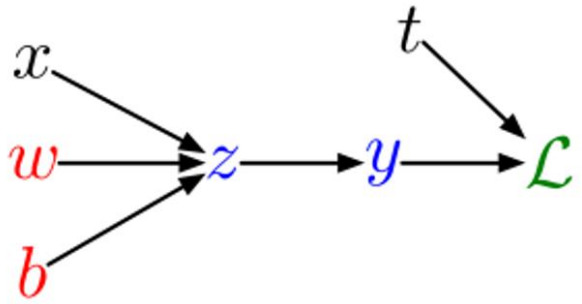
Instead, let's work *backwards*, systematically from the loss, computing each intermediate derivative as we go:

Computing the loss:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Computing the derivatives:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} = y - t$$
$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} \cdot \sigma'(z)$$
$$\frac{\partial\mathcal{L}}{\partial w} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} \cdot x$$
$$\frac{\partial\mathcal{L}}{\partial b} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z}$$



Remember, the goal isn't to obtain closed-form solutions, but to be able to write a program that efficiently computes the derivatives.

A simple non-linear regression example

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Instead, let's work *backwards*, systematically from the loss, computing each intermediate derivative as we go:

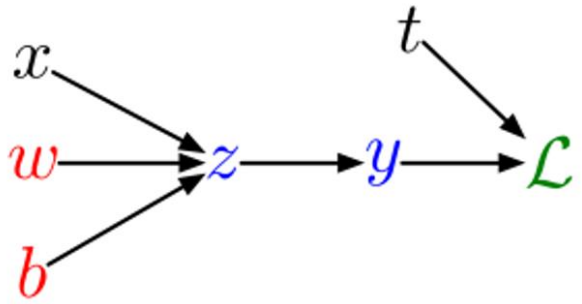
Computing the loss:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Do you see any structure here that could lead to a general algorithm?

Computing the derivatives:

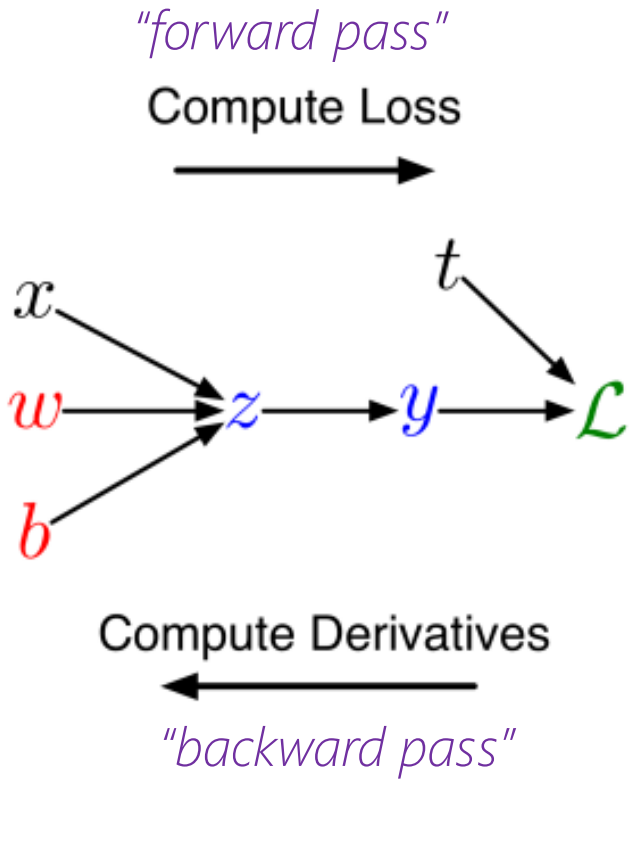
$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} = y - t$$
$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} \cdot \sigma'(z)$$
$$\frac{\partial\mathcal{L}}{\partial w} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} \cdot x$$
$$\frac{\partial\mathcal{L}}{\partial b} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z}$$



Remember, the goal isn't to obtain closed-form solutions, but to be able to write a program that efficiently computes the derivatives.

Systematizing derivatives with computation graphs

- We can diagram out the computations using a computation graph.
- The nodes represent all the inputs and computed quantities, and the edges represent which nodes are computed directly as a function of which other nodes.



$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

$$\frac{d\mathcal{L}}{dy} = y - t$$
$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy}\,\sigma'(z)$$
$$\frac{\partial\mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz}\cdot x$$
$$\frac{\partial\mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}$$

A slightly more convenient notation, use the bar, e.g.,"$\bar{z}$" to denote derivative of loss wrt that variable, $\frac{d\mathcal{L}}{dz}$, "error signal":

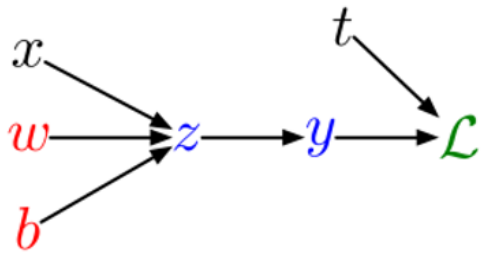
$$\bar{y} = y - t$$
$$\bar{z} = \bar{y}\cdot\sigma'(z)$$
$$\bar{w} = \bar{z}\cdot x$$
$$\bar{b} = \bar{z}$$

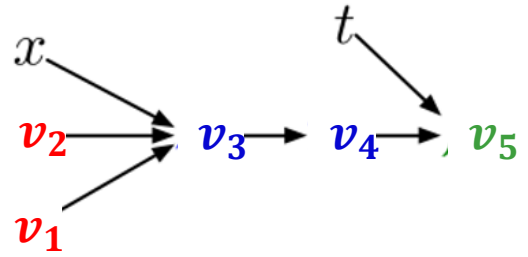
Systematizing derivatives with computation graphs

Backpropagation algorithm for single-child graphs:

Let $v_1, \ldots, v_N$ be a topological ordering of the computation graph (i.e. parents come before children.)

v_N denotes the variable we're trying to compute derivatives of (e.g. loss).





Computing the loss:

$v_3 = z = wx + b$
$v_4 = y = \sigma(z)$
$v_5 = \mathcal{L} = \frac{1}{2}(y - t)^2$

forward pass

For $i = 1, \ldots, N$

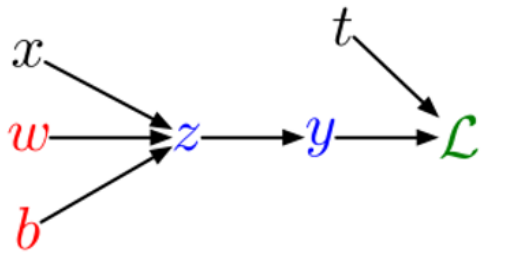
Compute v_i as a function of $\mathrm{Pa}(v_i)$

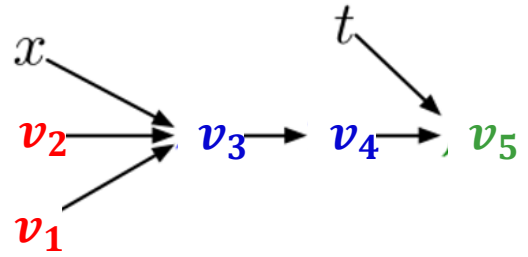
Systematizing derivatives with computation graphs

Backpropagation algorithm for single-child graphs:

Let $v_1, \ldots, v_N$ be a topological ordering of the computation graph (i.e. parents come before children.)

v_N denotes the variable we're trying to compute derivatives of (e.g. loss).





Computing the loss:

$$v_3 = z = wx + b$$
$$v_4 = y = \sigma(z)$$
$$v_5 = \mathcal{L} = \frac{1}{2}(y - t)^2$$

forward pass

For $i = 1, \ldots, N$

Compute v_i as a function of $\mathrm{Pa}(v_i)$

Computing the derivatives:

$$\overline{v_4} = \overline{y} = y - t$$
$$\overline{v_3} = \overline{z} = \overline{y} \cdot \sigma'(z)$$
$$\overline{v_2} = \overline{w} = \overline{z} \cdot x$$
$$\overline{v_1} = \overline{b} = \overline{z}$$

backward pass

$$\overline{v_N} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \ldots, 1$

?

Computing the derivatives:

$$\overline{v_4} = \overline{v_5}\frac{\partial v_5}{\partial v_4} \qquad \frac{d\mathcal{L}}{dy} = y - t$$

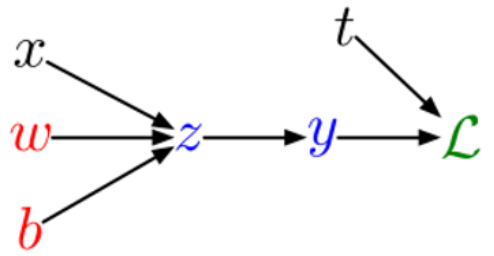
$$\overline{v_3} = \overline{v_4}\frac{\partial v_4}{\partial v_3} \qquad \frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy}\,\sigma'(z)$$

$$\overline{v_2} = \overline{v_3}\frac{\partial v_3}{\partial v_2} \qquad \frac{\partial \mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz} \cdot x$$

$$\overline{v_1} = \overline{v_3}\frac{\partial v_3}{\partial v_1} \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}$$

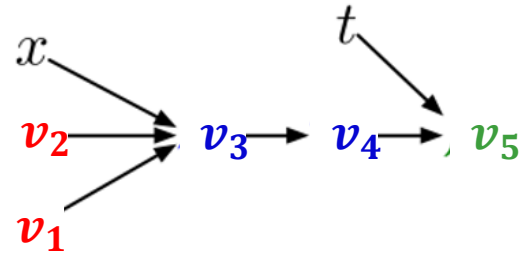
Systematizing derivatives with computation graphs

Backpropagation algorithm for single-child graphs:



Let $v_1, \ldots, v_N$ be a topological ordering of the computation graph (i.e. parents come before children.)

v_N denotes the variable we're trying to compute derivatives of (e.g. loss).



Computing the loss:

$$v_3 = z = wx + b$$
$$v_4 = y = \sigma(z)$$
$$v_5 = \mathcal{L} = \frac{1}{2}(y - t)^2$$

forward pass

For $i = 1, \ldots, N$

Compute v_i as a function of $\mathrm{Pa}(v_i)$

Computing the derivatives:

$$\overline{v_4} = \overline{y} = y - t$$
$$\overline{v_3} = \overline{z} = \overline{y} \cdot \sigma'(z)$$
$$\overline{v_2} = \overline{w} = \overline{z} \cdot x$$
$$\overline{v_1} = \overline{b} = \overline{z}$$

backward pass

$$\overline{v_N} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \ldots, 1$

$$\overline{v_i} := \frac{\partial \mathcal{L}}{\partial v_i} = \overline{v_{child(i)}} \frac{\partial v_{child(i)}}{\partial v_i}$$

Computing the derivatives:

$$\overline{v_4} = \overline{v_5} \frac{\partial v_5}{\partial v_4} \qquad \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} = y - t$$

$$\overline{v_3} = \overline{v_4} \frac{\partial v_4}{\partial v_3} \qquad \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} \cdot \sigma'(z)$$

$$\overline{v_2} = \overline{v_3} \frac{\partial v_3}{\partial v_2} \qquad \frac{\partial \mathcal{L}}{\partial w} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} \cdot x$$

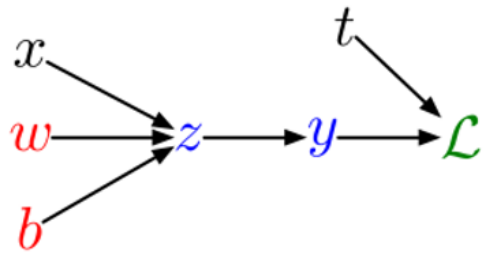
$$\overline{v_1} = \overline{v_3} \frac{\partial v_3}{\partial v_1} \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z}$$

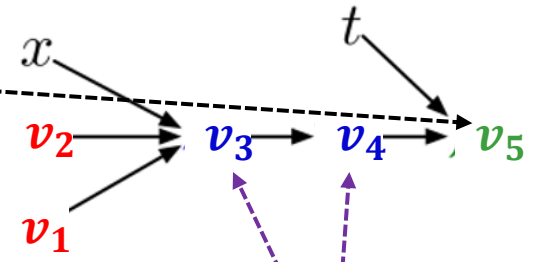
Systematizing derivatives with computation graphs

So why do I need the forward pass?

i. to compute derivatives

ii. to track the loss during training





Computing the loss:

$$v_3 = z = wx + b$$
$$v_4 = y = \sigma(z)$$
$$v_5 = \mathcal{L} = \frac{1}{2}(y - t)^2$$

forward pass

For $i = 1, \ldots, N$

Compute v_i as a function of $\mathrm{Pa}(v_i)$

Computing the derivatives:

$$\overline{v_4} = \bar{y} = y - t$$
$$\overline{v_3} = \bar{z} = \bar{y} \cdot \sigma'(z)$$
$$\overline{v_2} = \overline{w} = \bar{z} \cdot x$$
$$\overline{v_1} = \bar{b} = \bar{z}$$

backward pass

$$\overline{v_N} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \ldots, 1$

$$\overline{v_i} := \frac{\partial \mathcal{L}}{\partial v_i} = \overline{v_{child(i)}} \frac{\partial v_{child(i)}}{\partial v_i}$$

Computing the derivatives:

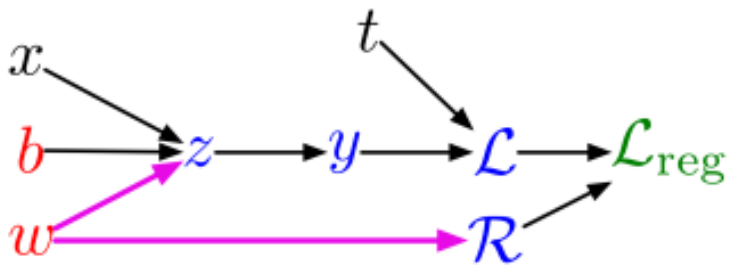
$$\overline{v_4} = \overline{v_5} \frac{\partial v_5}{\partial v_4}$$
$$\overline{v_3} = \overline{v_4} \frac{\partial v_4}{\partial v_3}$$
$$\overline{v_2} = \overline{v_3} \frac{\partial v_3}{\partial v_2}$$
$$\overline{v_1} = \overline{v_3} \frac{\partial v_3}{\partial v_1}$$

$$\frac{d\mathcal{L}}{dy} = y - t$$
$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy} \cdot \sigma'(z)$$
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz} \cdot x$$
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}$$

Problem: what if neural network has multiple children?

Problem: what if the computation graph has fan-out > 1?
This requires the multivariate Chain Rule!

L_2-Regularized regression

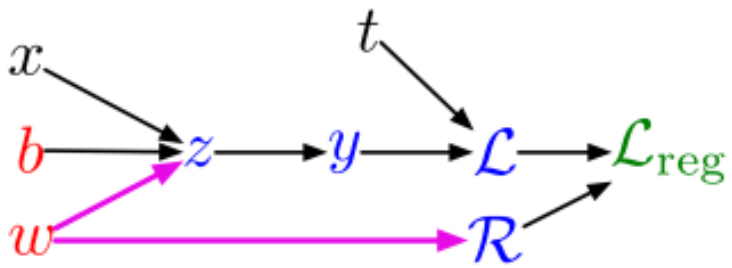


$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$
$$\mathcal{R} = \frac{1}{2}w^2$$
$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$$

Problem: what if neural network has multiple children?

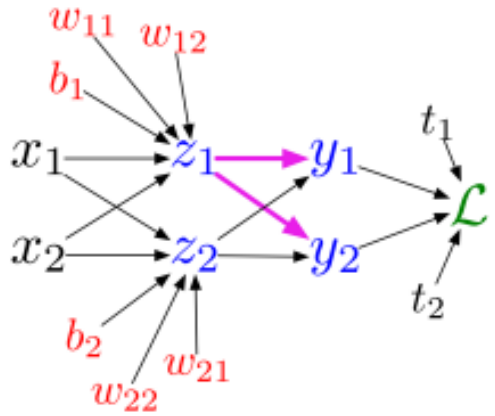
Problem: what if the computation graph has fan-out > 1?
This requires the multivariate Chain Rule!

L_2-Regularized regression



$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$
$$\mathcal{R} = \frac{1}{2}w^2$$
$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$$

Multclass logistic regression

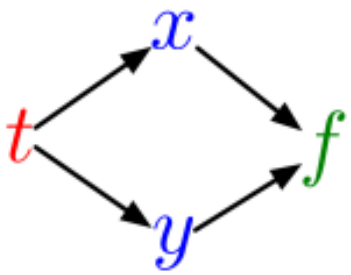


$$z_\ell = \sum_j w_{\ell j} x_j + b_\ell$$
$$y_k = \frac{e^{z_k}}{\sum_\ell e^{z_\ell}}$$
$$\mathcal{L} = -\sum_k t_k \log y_k$$

Calculus: Multivariate chain rule

- Suppose we have a function $f(x, y)$ and functions $x(t)$ and $y(t)$. (All the variables here are scalar-valued.) Then

$$\frac{\mathrm{d}}{\mathrm{d}t} f(x(t), y(t)) = \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}$$



total effect of t on the f is the sum of all the separate paths through which it exerts influence

- Example:

$$f(x, y) = y + e^{xy}$$
$$x(t) = \cos t$$
$$y(t) = t^2$$

- Plug in to Chain Rule:

$$\frac{\mathrm{d}f}{\mathrm{d}t} = \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}$$
$$= (ye^{xy}) \cdot (-\sin t) + (1 + xe^{xy}) \cdot 2t$$

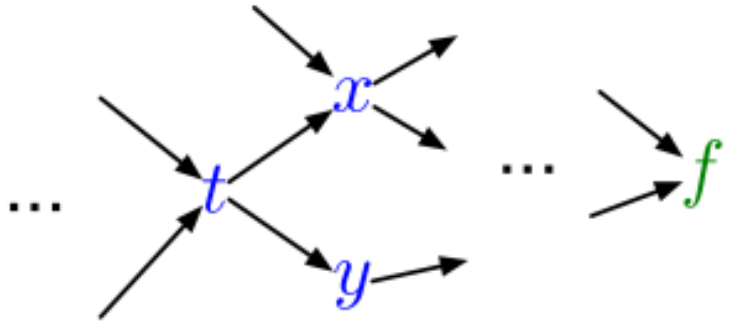
Calculus: Multivariate chain rule

- In the context of backpropagation:

Mathematical expressions to be evaluated

$$\frac{\mathrm{d}f}{\mathrm{d}t} = \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}$$

"Messages" (values) already computed by our program



- In our notation:

recall: $\bar{z} \doteq \frac{d\mathcal{L}}{dz}$

$$\bar{t} = \bar{x} \cdot \frac{\mathrm{d}x}{\mathrm{d}t} + \bar{y} \cdot \frac{\mathrm{d}y}{\mathrm{d}t}$$

The general backpropagation algorithm

(including for multi-child graphs)

Let $v_1, \ldots, v_N$ be a topological ordering of the computation graph (i.e. parents come before children.)

v_N denotes the variable we're trying to compute derivatives of (e.g. loss).

Computing the loss:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

forward pass

For $\frac{\mathrm{d}f}{\mathrm{d}t} = \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}$

Compute v_i as a function of $\mathrm{Pa}(v_i)$

Computing the derivatives:

$$\overline{v_4} = \overline{y} = y - t$$
$$\overline{v_3} = \overline{z} = \overline{y} \cdot \sigma'(z)$$
$$\overline{v_2} = \overline{w} = \overline{z} \cdot x$$
$$\overline{v_1} = \overline{b} = \overline{z}$$

backward pass

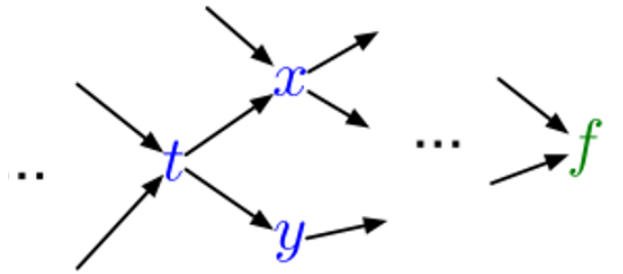
$$\overline{v_N} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \ldots, 1$

$$\overline{v_i} = \sum_{j \in \mathrm{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$$

$$\overline{v_i} = \frac{\partial \mathcal{L}}{\partial v_i} = \overline{v_{i+1}} \frac{\partial v_{i+1}}{\partial v_i}$$

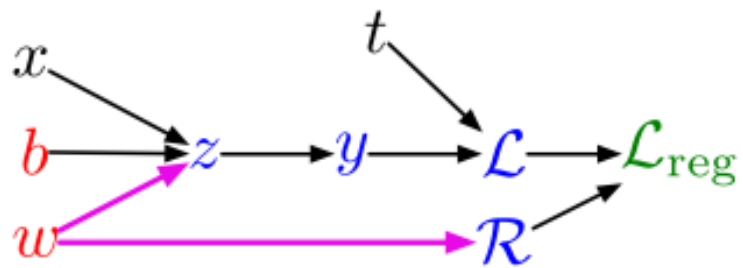
$$\frac{\mathrm{d}f}{\mathrm{d}t} = \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}$$



$$\overline{t} = \overline{x} \cdot \frac{\mathrm{d}x}{\mathrm{d}t} + \overline{y} \cdot \frac{\mathrm{d}y}{\mathrm{d}t}$$

The general backpropagation algorithm

Example: univariate logistic least squares regression

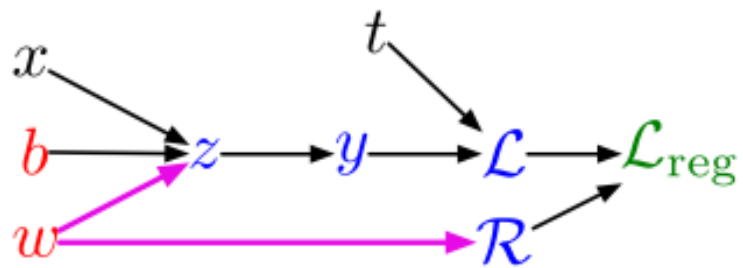


Forward pass:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$
$$\mathcal{R} = \frac{1}{2}w^2$$
$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$$

The general backpropagation algorithm

Example: univariate logistic least squares regression



Forward pass:

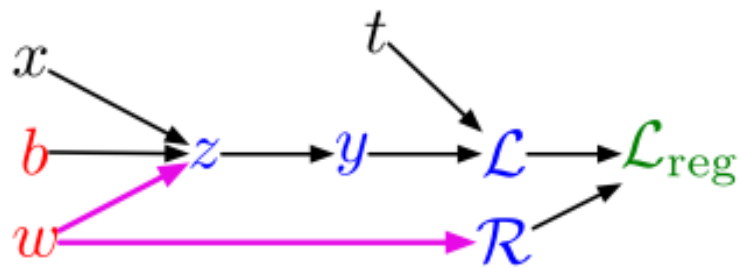
$$
\begin{aligned}
z &= wx + b \\
y &= \sigma(z) \\
\mathcal{L} &= \frac{1}{2}(y - t)^2 \\
\mathcal{R} &= \frac{1}{2}w^2 \\
\mathcal{L}_{\text{reg}} &= \mathcal{L} + \lambda\mathcal{R}
\end{aligned}
$$

Backward pass: $\overline{v_i} = \sum_{j \in \text{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$

$$
\begin{aligned}
\overline{\mathcal{L}_{\text{reg}}} &= 1 \\
\overline{\mathcal{R}} &= \overline{\mathcal{L}_{\text{reg}}} \frac{\mathrm{d}\mathcal{L}_{\text{reg}}}{\mathrm{d}\mathcal{R}} \\
&= \overline{\mathcal{L}_{\text{reg}}}\,\lambda \\
\overline{\mathcal{L}} &= \overline{\mathcal{L}_{\text{reg}}} \frac{\mathrm{d}\mathcal{L}_{\text{reg}}}{\mathrm{d}\mathcal{L}} \\
&= \overline{\mathcal{L}_{\text{reg}}} \\
\overline{y} &= \overline{\mathcal{L}} \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} \\
&= \overline{\mathcal{L}}\,(y - t)
\end{aligned}
$$

The general backpropagation algorithm

Example: univariate logistic least squares regression



Forward pass:

$$
\begin{aligned}
z &= wx + b \\
y &= \sigma(z) \\
\mathcal{L} &= \frac{1}{2}(y - t)^2 \\
\mathcal{R} &= \frac{1}{2}w^2 \\
\mathcal{L}_{\text{reg}} &= \mathcal{L} + \lambda\mathcal{R}
\end{aligned}
$$

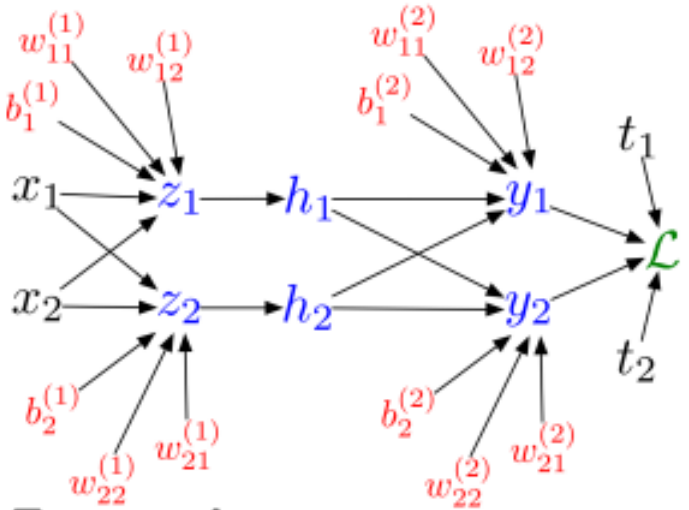
Backward pass: $\overline{v_i} = \sum_{j \in \text{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$

$$
\begin{aligned}
\overline{\mathcal{L}_{\text{reg}}} &= 1 \\
\overline{\mathcal{R}} &= \overline{\mathcal{L}_{\text{reg}}} \frac{\mathrm{d}\mathcal{L}_{\text{reg}}}{\mathrm{d}\mathcal{R}} \\
&= \overline{\mathcal{L}_{\text{reg}}}\,\lambda \\
\overline{\mathcal{L}} &= \overline{\mathcal{L}_{\text{reg}}} \frac{\mathrm{d}\mathcal{L}_{\text{reg}}}{\mathrm{d}\mathcal{L}} \\
&= \overline{\mathcal{L}_{\text{reg}}} \\
\overline{y} &= \overline{\mathcal{L}} \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} \\
&= \overline{\mathcal{L}}\,(y - t)
\end{aligned}
$$

$$
\begin{aligned}
\overline{z} &= \overline{y} \frac{\mathrm{d}y}{\mathrm{d}z} \\
&= \overline{y}\,\sigma'(z) \\
\overline{w} &= \overline{z} \frac{\partial z}{\partial w} + \overline{\mathcal{R}} \frac{\mathrm{d}\mathcal{R}}{\mathrm{d}w} \\
&= \overline{z}\,x + \overline{\mathcal{R}}\,w \\
\overline{b} &= \overline{z} \frac{\partial z}{\partial b} \\
&= \overline{z}
\end{aligned}
$$

The general backpropagation algorithm

Multilayer Perceptron (multiple outputs):



Forward pass:

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

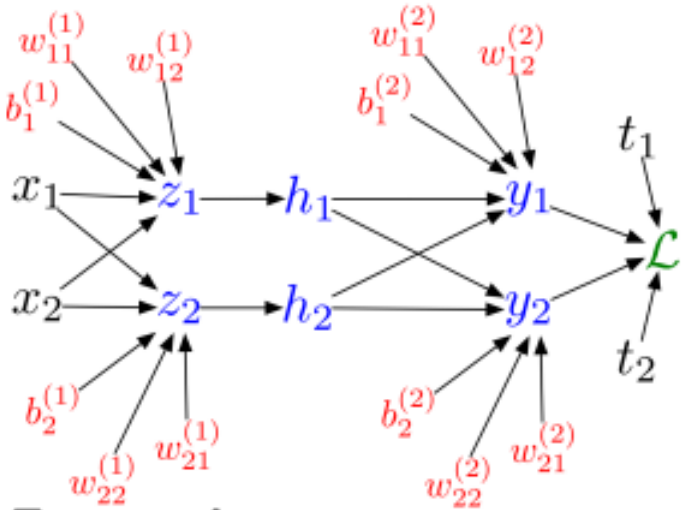
$$h_i = \sigma(z_i)$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \sum_k (y_k - t_k)^2$$

The general backpropagation algorithm

Multilayer Perceptron (multiple outputs):



Forward pass:

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i = \sigma(z_i)$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \sum_k (y_k - t_k)^2$$

Backward pass: $\overline{v_i} = \sum_{j \in \mathrm{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$

$$\overline{\mathcal{L}} = 1$$

$$\overline{y_k} = \overline{\mathcal{L}} \ \ (-1)(t_k - y_k)$$

$$\overline{w_{ki}^{(2)}} = \overline{y_k}\, h_i$$

$$\overline{b_k^{(2)}} = \overline{y_k}$$

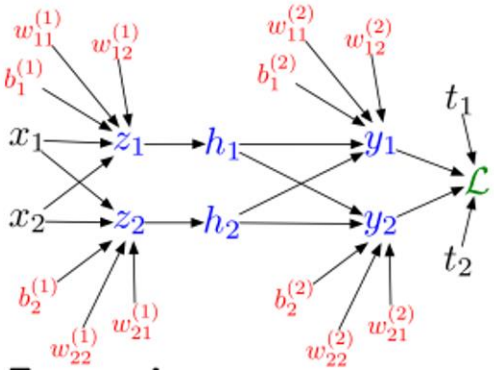
$$\overline{h_i} = \sum_k \overline{y_k} w_{ki}^{(2)}$$

$$\overline{z_i} = \overline{h_i}\, \sigma'(z_i)$$

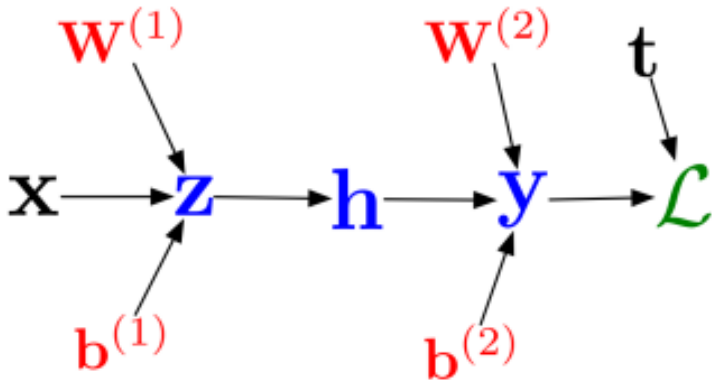
$$\overline{w_{ij}^{(1)}} = \overline{z_i}\, x_j$$

$$\overline{b_i^{(1)}} = \overline{z_i}$$

Again, but in vector notation



In vectorized form:



Forward pass:

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)} \qquad \mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$h_i = \sigma(z_i) \qquad \mathbf{h} = \sigma(\mathbf{z})$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)} \qquad \mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\mathcal{L} = \frac{1}{2}\sum_k (y_k - t_k)^2 \qquad \mathcal{L} = \frac{1}{2}\|\mathbf{t} - \mathbf{y}\|^2$$

$$\overline{v_i} = \sum_{j \in \mathrm{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$$

Backward pass:

$$\overline{\mathcal{L}} = 1$$

$$\overline{\mathbf{y}} = \overline{\mathcal{L}}\ (-1)(\boldsymbol{t} - \boldsymbol{y})$$

$$\overline{\mathbf{W}^{(2)}} = \overline{\mathbf{y}}\mathbf{h}^\top$$

$$\overline{\mathbf{b}^{(2)}} = \overline{\mathbf{y}}$$

$$\overline{\mathbf{h}} = \mathbf{W}^{(2)\top}\overline{\mathbf{y}}$$

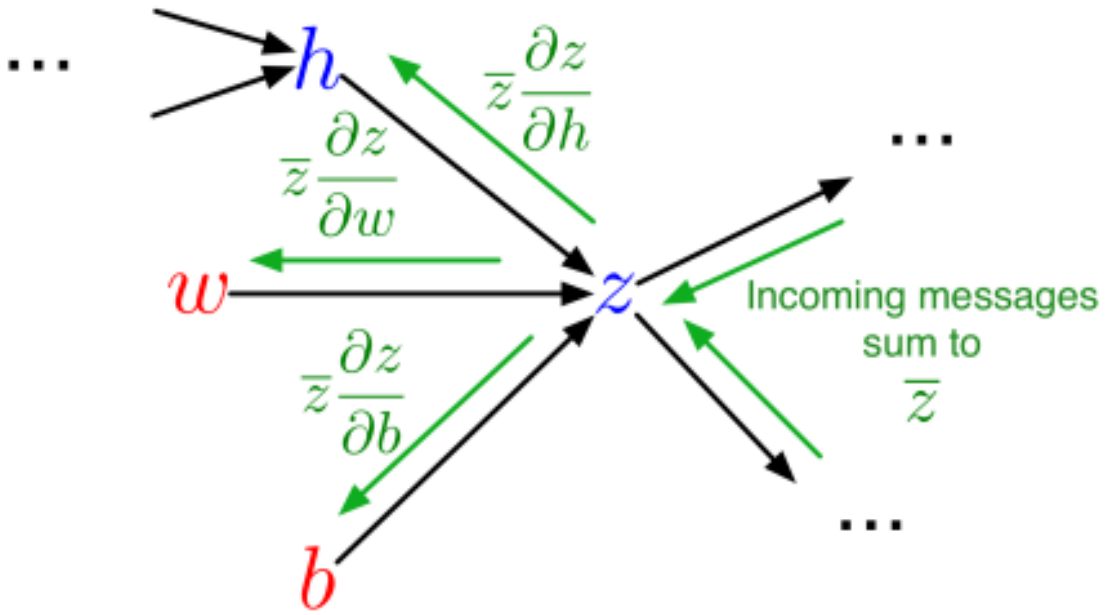
$$\overline{\mathbf{z}} = \overline{\mathbf{h}} \cdot \sigma'(\mathbf{z})$$

$$\overline{\mathbf{W}^{(1)}} = \overline{\mathbf{z}}\mathbf{x}^\top$$

$$\overline{\mathbf{b}^{(1)}} = \overline{\mathbf{z}}$$

Backpropagation as message passing

$$\overline{v_i} = \sum_{j \in \mathrm{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$$



- Each node receives a bunch of messages from its children, which it aggregates to get its error signal. It then passes messages to its parents.
- This provides modularity, since each node only has to know how to compute derivatives with respect to its arguments, and doesn't have to know anything about the rest of the graph.

Backprop: computational cost

- Computational cost of forward pass: one multiply operation per weight, $w_{i,j}$

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$w_{i,j}$ is used only once (one term in the sum): to compute information from x_j to z_i

- Computational cost of backward pass: two multiply operations per weight, $w_{k,i}$

$$\overline{w_{ki}^{(2)}} = \overline{y_k}\, h_i$$

1. computing the weight's own error signal

$$\overline{h_i} = \sum_k \overline{y_k} w_{ki}^{(2)}$$

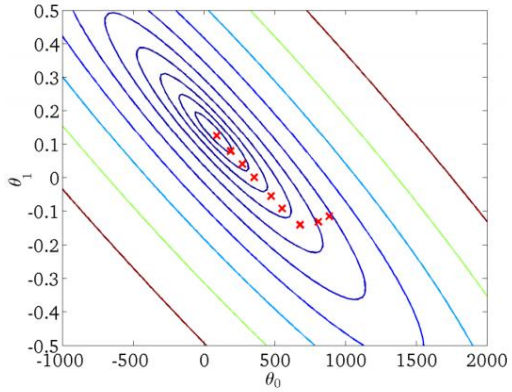
2. propagating that error signal backward

t as expensive as two

- Linear in # of parameters!
- Linear in number of inputs!
- Linear in number of hidden nodes!

Three views of the gradient

- **Geometric:** visualization of gradient in weight space
- **Algebraic:** mechanics of computing the derivatives
- **Implementational:** efficient implementation on the computer



$$\mathcal{L} = \frac{1}{2}(\sigma(wx+b)-t)^2$$

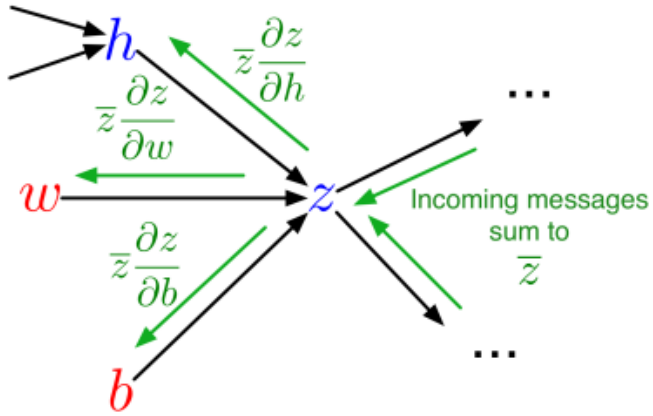
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\frac{1}{2}(\sigma(wx+b)-t)^2\right]$$

$$= \frac{1}{2}\frac{\partial}{\partial w}(\sigma(wx+b)-t)^2$$

$$= (\sigma(wx+b)-t)\frac{\partial}{\partial w}(\sigma(wx+b)-t)$$

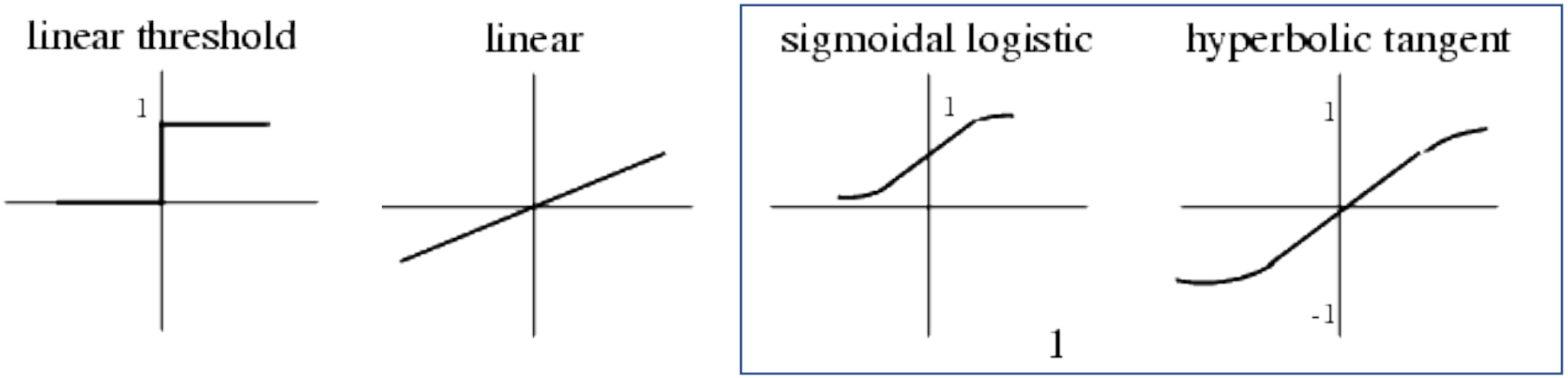
$$= (\sigma(wx+b)-t)\sigma'(wx+b)\frac{\partial}{\partial w}(wx+b$$

$$= (\sigma(wx+b)-t)\sigma'(wx+b)x$$



Each one is important to understand, as well as the connections between them.

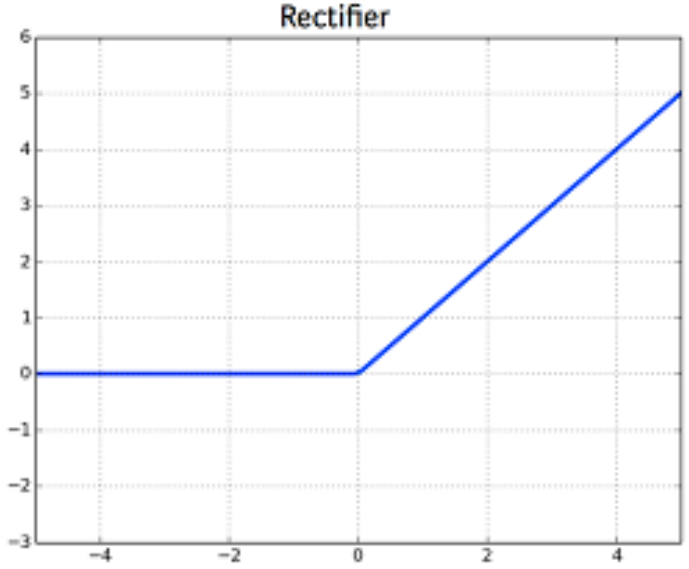
Back to activation functions



- What's the common issue wrt sigmoidal/hyberbolic tangent activations and gradient descent?
- These functions have asymptotes at at the top and bottom, and thus have zero gradient there. Easy to get "stuck" with zero gradients—can become "dead units".

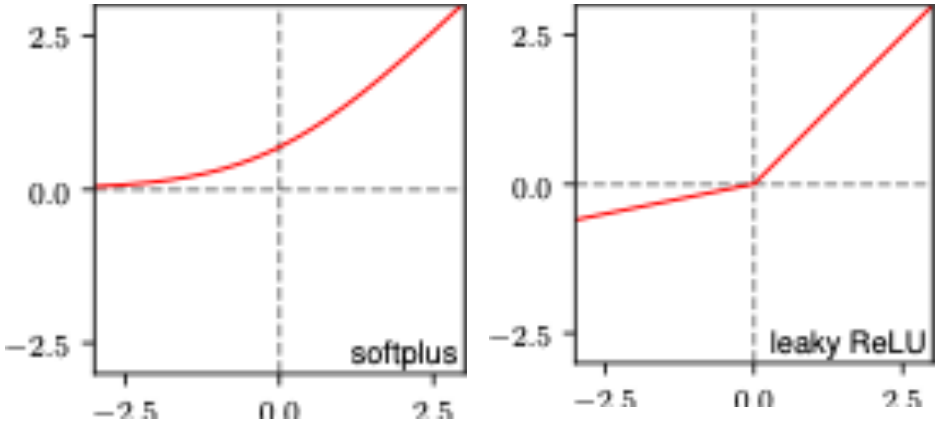
ReLUs to the (half) rescue

Rectified Linear Unit



$$RELU(x) = \begin{cases} 0 \text{ if } x < 0 \\ x \text{ if } x >= 0 \end{cases}$$

- ReLU gets rid of half this problem ($x > 0$).
- Still have dead units when $x < 0$, why is this catastrophic?
- Many tricks to reduce # of dead units, such as smaller learning rate, "batch normalization", tweaking ReLU to be "leaky".



Could we use finite difference instead?

Computing the **gradient** using numerical differentiation:

$$\frac{\partial}{\partial w_i} E(\mathrm{w}; \mathcal{D}) \approx \frac{E(\mathrm{w} + \epsilon\, I_i; \mathcal{D}) - E(\mathrm{w} - \epsilon\, I_i; \mathcal{D})}{2\epsilon}$$

for a D dimensional weight vector $\mathrm{w} \in \mathbb{R}^D$ requires:

$$\left[\frac{\partial}{\partial w_1} E(\mathrm{w}; \mathcal{D}), \ldots, \frac{\partial}{\partial w_D} E(\mathrm{w}; \mathcal{D})\right]$$

$2D$ error calculations. Each **error calculation** requires $\mathcal{O}(ND)$ computation for N datapoints.

- **Total cost:** $\mathcal{O}(ND^2)$ per gradient step! (Quadratic in # of parameters)
  - For real problems (minibatch) often $N \approx 1000$ and $D \gg 10^6$ → $ND^2 \gg 10^{3+2*6} = 10^{15}$
  - Stochastic gradient with batch size 1: $D^2 \gg 10^{12}$

Could we use symbolic algebra systems?

Computer algebra systems like SymPy can be used to automate the differentiation process.

- Express functions **symbolically**
- Automatically construct **derivative expressions**

Advantages:

- **Automatic:** algebraic systems can be applied to "any" expression
  - Some issues with error functions that contain control flow/loops.
- **Exact:** Doesn't require epsilon approximation

Disadvantages:

- **Inefficient:** algebraic derivative expressions could be significantly larger and contain many repeated expressions.

However, in the modern era, we have "auto-differentiation", which performs backpropagation automatically, computing derivative as needed, rather than manually.

So why are DNNs so good?!

- Linear regression with (arbitrarily high order) polynomial features are also universal function approximators.
- So why do DNNs dominate empirically sometimes?
- Not fully understood—something to do with the optimization landscape created by these massive architectures.
- Active area of research among theorists.

So why are DNNs so good?!

- Linear reg
  polynomi
  approxim
- So why d
- Not fully
  optimizat
  architectu
- Active are

Deep Learning as a Magic Mirror

David Donoho Every theorist who looks at it see what they wish

https://stats385.github.io/assets/lectures/StanfordStats385-20170927-Lecture01-Donoho.pdf?fbclid=IwAR3xBtqNW_8-JJ8zkbCm-tIJQD9B6O4uPFR4oQied3iBYyhzNkFeyj6bJM4